

components generate and manipulate DOM in the browser by default. However, the same components may be rendered into HTML strings on the server, sent directly to the browser, and then “hydrated” into a fully interactive app on the client.

A server-rendered Vue.js app is often known as a “isomorphic” or “universal” app since the bulk of the code operates on both the server and the client.

What is the significance of SSR?

The primary benefit of SSR over a standard SPA (Single-Page Application) is improved SEO, as search engine crawlers can see the completely made page.

It's worth noting that for the time being, Google and Bing index synchronous JavaScript applications just as well. The main term here is synchronization. The crawler will not pause for you to finish if your app begins with a loading spinner and then fetches content using Ajax. This implies that if you have asynchronously fetched content on pages where SEO is relevant, SSR will be needed.

- **Faster time-to-content, particularly on slow internet or devices**

Since server-rendered markup may not have to wait for all JavaScript to be downloaded and performed before being shown, the customer can see a fully rendered page earlier. This usually leads to a stronger user experience, which is especially important in applications where time-to-content is directly related to conversion performance.

When using SSR, there are several trade-offs to consider:

- **Constraints on development**

Browser-specific code may only be found inside specific lifecycle hooks; certain additional libraries can need special care to operate in a server-rendered app.

- **Create setup and deployment specifications that are more involved**

Unlike a fully static SPA that can be implemented on any static file server, a server-rendered app needs an environment where a Node.js server can run.

- **Increased server-side load**

Rendering a complete app in Node.js is clearly going to be more CPU-